Question 1

1.1

In [1]:
```
#Importing Libraries
import json
import re
import pandas as pd
import numpy as np
```

In [2]:
```
#Loading json file
with open("research_abstracts.json", "r", encoding = "utf-8") as e:
    raw_data = json.load(e)

data = pd.DataFrame(raw_data)
print("Shape:",data.shape)
data.head()
```

```
Shape: (1138, 3)
```

Out[2]:

	DOI	Abstract	Label
0	10.1155/2021/1071145	Forward-looking forecasting of the inflation r...	ai
1	10.1007/s43546-022-00384-2	Our study examined the disaggregation of infla...	ai
2	10.1002/for.2667	This paper develops a dynamic factor model tha...	classical
3	10.1007/s42973-021-00091-x	In this paper, I use a simple SIR Macro model ...	classical
4	10.1007/s40258-012-0003-z	BackgroundGame theory is useful for identifyin...	classical

In [3]:
```
#Searching for the number of null values, and duplicates in DOI and Abstract
print("Missing Values:\n", data.isnull().sum())
print("Duplicate DOI:", data["DOI"].duplicated().sum())
```

```
print("Dupplicate Abstract:", data["Abstract"].duplicated().sum())
print("Label distribution:", data["Label"].value_counts())
```

```
Missing Values:
 DOI         4
Abstract    0
Label       0
dtype: int64
Duplicate DOI: 3
Dupplicate Abstract: 13
Label distribution: Label
classical    900
ai           238
Name: count, dtype: int64
```

In [4]:
```
data = data.drop_duplicates(subset = "Abstract").reset_index(drop = True)
data = data.dropna(subset=["DOI"])
print(data.shape)
data.head()
```

```
(1123, 3)
```

Out[4]:

	DOI	Abstract	Label
0	10.1155/2021/1071145	Forward-looking forecasting of the inflation r...	ai
1	10.1007/s43546-022-00384-2	Our study examined the disaggregation of infla...	ai
2	10.1002/for.2667	This paper develops a dynamic factor model tha...	classical
3	10.1007/s42973-021-00091-x	In this paper, I use a simple SIR Macro model ...	classical
4	10.1007/s40258-012-0003-z	BackgroundGame theory is useful for identifyin...	classical

In [5]:
```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
```

In [6]:
```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()
```

```
def denoise_text(text):
    #for lowercase, remove URLs, keep letters only, normalise whitespaces
    #tokenise, lemmatise and stop word
    text = text.lower()
    text = re.sub(r"http\S+|www\.\S+", " ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens if t not in stop_words and len(t) > 2]
    return " ".join(tokens)

data["Clean_Abstract"] = data["Abstract"].apply(denoise_text)
data[["Abstract", "Clean_Abstract"]].head(5)
```

Out[6]:

	Abstract	Clean_Abstract
0	Forward-looking forecasting of the inflation r...	forward looking forecasting inflation rate cou...
1	Our study examined the disaggregation of infla...	study examined disaggregation inflation compon...
2	This paper develops a dynamic factor model tha...	paper develops dynamic factor model us euro ar...
3	In this paper, I use a simple SIR Macro model ...	paper use simple sir macro model examine japan...
4	BackgroundGame theory is useful for identifyin...	backgroundgame theory useful identifying condi...

1.2

In [7]:

```
from sklearn.feature_extraction.text import TfidfVectorizer

X_text = data["Clean_Abstract"]
y = data["Label"]

tfidf = TfidfVectorizer(max_features = 3000, ngram_range = (1, 2), min_df = 2)
X = tfidf.fit_transform(X_text)

print("TF-IDF matrix shape:", X.shape)
print("Sample vocabulary:", list(tfidf.get_feature_names_out()[:12]))
```

```
TF-IDF matrix shape: (1123, 3000)
Sample vocabulary: ['ability', 'able', 'absence', 'absolute', 'absolute error', 'absolute percentage', 'abstract', 'academic',
'access', 'accommodate', 'accordance', 'according']
```

1.3

In [8]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state = 42, stratify = y)

print("Train size:", X_train.shape[0], " Test size:", X_test.shape[0])
print("\nTrain label distribution:\n", y_train.value_counts())
print("\nTest Label distribution:\n", y_test.value_counts())
```

```
Train size: 898  Test size: 225

Train label distribution:
 Label
classical    712
ai           186
Name: count, dtype: int64

Test Label distribution:
 Label
classical    178
ai            47
Name: count, dtype: int64
```

Baseline Model

In [9]:
```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

clf = MultinomialNB()
clf.fit(X_train, y_train)

train_pred = clf.predict(X_train)
test_pred = clf.predict(X_test)
```

```
train_acc = accuracy_score(y_train, train_pred)
test_acc = accuracy_score(y_test, test_pred)

print(f"Training Accuracy: {train_acc:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
```

```
Training Accuracy: 0.9109
Test Accuracy: 0.8756
```

Performance report

In [10]:
```
print("Classification report (test set):\n")
print(classification_report(y_test, test_pred))

print("Confusion Matrix (rows = actual, cols = predicted, order = [ai, classical]):")
print(confusion_matrix(y_test, test_pred, labels = ["ai", "classical"]))
```

```
Classification report (test set):

              precision    recall  f1-score   support

          ai       0.95      0.43      0.59        47
   classical       0.87      0.99      0.93       178

    accuracy                           0.88       225
   macro avg       0.91      0.71      0.76       225
weighted avg       0.89      0.88      0.86       225

Confusion Matrix (rows = actual, cols = predicted, order = [ai, classical]):
[[ 20  27]
 [  1 177]]
```

Comment

The training accuracy is 91.03% and the test accuracy is 87.56% which are close to each other just3.47% apart. This is a small gap so the model is not badly overfitting as it has learnt genuine patterns in the vocabulary rather than memorising the training abstracts.

The accuracies can be misleading as we see a test accuracy of 87.56% which looks strong but the dataset is imbalanced with 900 classical records vs 238 that are ai which is 79% as to 21% record wise, the classifier would already score 79% accuracy because classical holds 79% of the records, so the 87.56% is a real but modest improvement over that naive baseline.

Recall for classical is 99% which is 177/178 correctly found, but the recall for ai is only 43% which is 20/47 correctly found. The confusion matrix shows 27 ai abstracts were misclassified as classical meaning the model is baised towards the majority class

Question 2

2.1

In [11]:
```
#Logistic Regression
from sklearn.linear_model import LogisticRegression

LR_Model = LogisticRegression(max_iter = 1000, random_state = 42)
LR_Model.fit(X_train, y_train)

y_pred_test = LR_Model.predict(X_test)
print("Classification Report:\n", classification_report(y_test, y_pred_test))
print("Confusion Matrix:\n", confusion_matrix(y_true = y_test, y_pred = y_pred_test))
print("Logistic Regression Test Accuracy: ", accuracy_score(y_test, y_pred_test))
```

```
Classification Report:
              precision    recall  f1-score   support

          ai       1.00      0.38      0.55        47
   classical       0.86      1.00      0.92       178

    accuracy                           0.87       225
   macro avg       0.93      0.69      0.74       225
weighted avg       0.89      0.87      0.85       225

Confusion Matrix:
 [[ 18  29]
 [  0 178]]
Logistic Regression Test Accuracy:  0.8711111111111111
```

In [12]:
```
#Linear SVM
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score


LinearSVM_Model = LinearSVC(random_state = 42)
LinearSVM_Model.fit(X_train, y_train)

y_pred_test = LinearSVM_Model.predict(X_test)
print("Classification Report:\n", classification_report(y_test, y_pred_test))
print("Confusion Matrix:\n", confusion_matrix(y_true = y_test, y_pred = y_pred_test))
print("Linear SVM Test Accuracy: ", accuracy_score(y_test, y_pred_test))
```

```
Classification Report:
              precision    recall  f1-score   support

          ai       0.89      0.66      0.76        47
   classical       0.92      0.98      0.95       178

    accuracy                           0.91       225
   macro avg       0.90      0.82      0.85       225
weighted avg       0.91      0.91      0.91       225

Confusion Matrix:
 [[ 31  16]
 [  4 174]]
Linear SVM Test Accuracy:  0.9111111111111111
```

In [13]:
```
#Random Forest Model
from sklearn.ensemble import RandomForestClassifier

RF_Model = RandomForestClassifier(n_estimators = 300, random_state = 42)
RF_Model.fit(X_train, y_train)

y_pred_test = RF_Model.predict(X_test)
print("Classification Report:\n", classification_report(y_test, y_pred_test))
print("Confusion Matrix:\n", confusion_matrix(y_true = y_test, y_pred = y_pred_test))
print("Random Forest Test Accuracy: ", accuracy_score(y_test, y_pred_test))
```

```
Classification Report:
              precision    recall  f1-score   support

          ai       1.00      0.68      0.81        47
   classical       0.92      1.00      0.96       178

    accuracy                           0.93       225
   macro avg       0.96      0.84      0.88       225
weighted avg       0.94      0.93      0.93       225

Confusion Matrix:
 [[ 32  15]
 [  0 178]]
Random Forest Test Accuracy:  0.9333333333333333
```

In [14]:
```
#XGBoost Model
from sklearn.ensemble import GradientBoostingClassifier

XGB_Model = GradientBoostingClassifier(random_state = 42)
XGB_Model.fit(X_train, y_train)

y_pred_test = XGB_Model.predict(X_test)
print("Classification Report:\n", classification_report(y_test, y_pred_test))
print("Confusion Matrix:\n", confusion_matrix(y_true = y_test, y_pred = y_pred_test))
print("XGBoost Test Accuracy: ", accuracy_score(y_test, y_pred_test))
```

```
Classification Report:
              precision    recall  f1-score   support

          ai       0.81      0.74      0.78        47
   classical       0.93      0.96      0.94       178

    accuracy                           0.91       225
   macro avg       0.87      0.85      0.86       225
weighted avg       0.91      0.91      0.91       225

Confusion Matrix:
 [[ 35  12]
 [  8 170]]
XGBoost Test Accuracy:  0.9111111111111111
```

Result

Three of the four models beat the baseline model. XGBoost had a test accuracy of 91%, Macro F1 of 86% and a ai recall of 74%. Random Forest had test accuracy of 93%, Macro F1 of 88% and ai recall of 68%. Linear SVM had a Test accuracy of 91%, Macro F1 of 85% and a ai recall of 66%.

Leaving Logistic Regression being the worst of the four models and even worser than the baseline as it got a test accuracy of 87,1% whilst the baseline test accuracy was 87,5%, it also performed below the baseline numbers on Macro F1 and ai recall as the baseline got Macro F1 of 76% and ai recall of 43% whilse Logistic Regression got Macro F1 of 75% and ai recall of 38%.

So the overall best Model is Random Forest as the model has a Test accuracy of 93% and a Macro F1 of 88%.

2.2

In [15]:
```
from sklearn.utils import resample
from collections import Counter
import matplotlib.pyplot as plt

print("Imbalanced class distribution:", Counter(y_train))
X_data = pd.DataFrame(X_train.toarray())
y_series = pd.Series(np.asarray(y_train), name = "y").reset_index(drop = True)
bal_data = pd.concat([X_data, y_series], axis = 1)
max_size = bal_data["y"].value_counts().max()
bal_upsampled = pd.concat([ resample(bal_data[bal_data["y"] == cls], replace = True,
                                     n_samples = max_size, random_state = 42)
                            for cls in bal_data["y"].unique()])

bal_upsampled = bal_upsampled.sample(frac = 1, random_state = 42).reset_index(drop = True)

X_balanced = bal_upsampled.drop(columns = ["y"]).values
y_balanced = bal_upsampled["y"]
print("Balanced class distribution:", Counter(y_balanced))
```

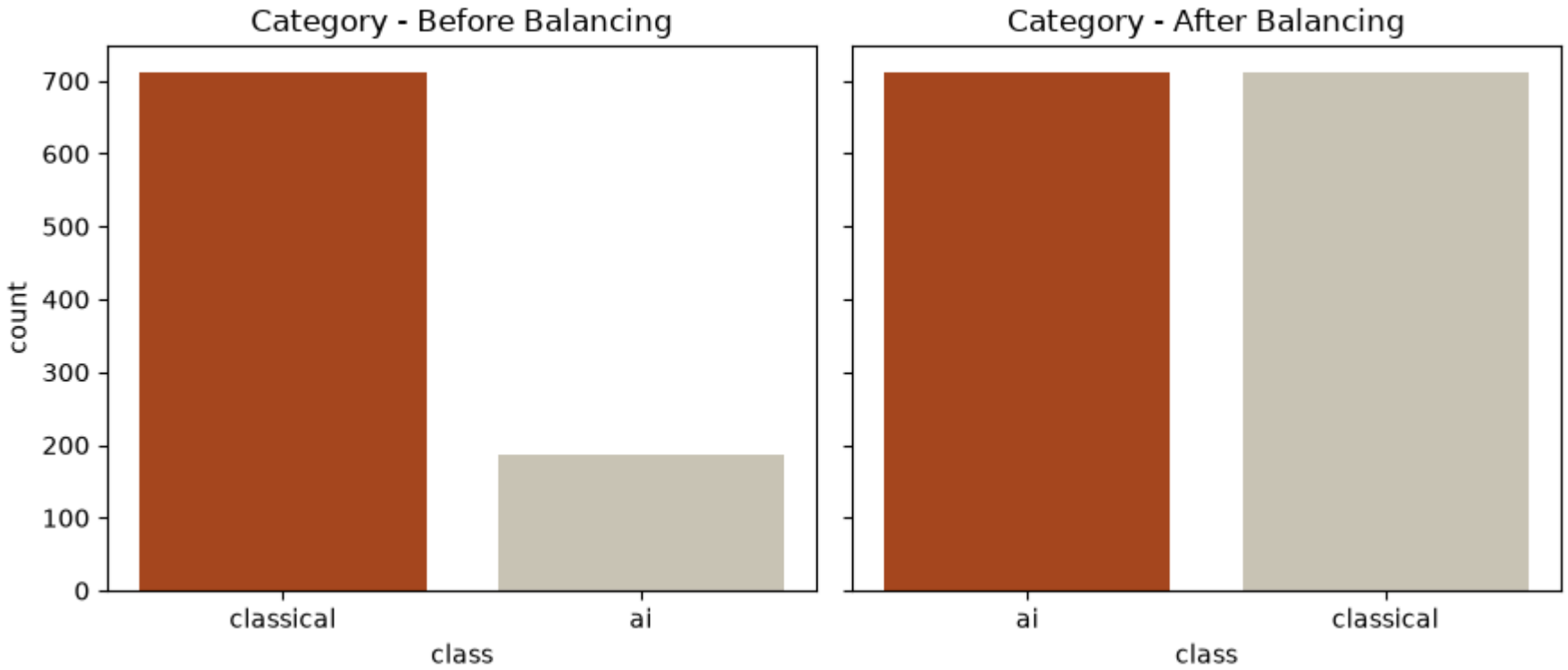
```
Imbalanced class distribution: Counter({'classical': 712, 'ai': 186})
Balanced class distribution: Counter({'ai': 712, 'classical': 712})
```

In [16]:
```
fig, axes = plt.subplots(1, 2, figsize = (9, 4), sharey = True)

count_before = pd.Series(y_train).value_counts()
count_after = pd.Series(y_balanced).value_counts()

axes[0].bar(count_before.index.astype(str), count_before.values, color = ["#a8481f", "#c9c3b6"])
axes[0].set_title("Category - Before Balancing")
axes[0].set_xlabel("class")
axes[0].set_ylabel("count")

axes[1].bar(count_after.index.astype(str), count_after.values, color = ["#a8481f", "#c9c3b6"])
axes[1].set_title("Category - After Balancing")
axes[1].set_xlabel("class")
plt.tight_layout()
plt.show()
```



In [17]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

type_labels = ["ai", "classical"]

max_epoch = 300
log_reg_clf = LogisticRegression(max_iter = max_epoch,
                                 random_state = 42)

log_reg_clf.fit(X_balanced, y_balanced)
y_pred_train_baseline = log_reg_clf.predict(X_train)
print("Training set\n", classification_report(y_train, y_pred_train_baseline, target_names = type_labels))
y_pred_test_baseline = log_reg_clf.predict(X_test)
print("Test set\n", classification_report(y_test, y_pred_test_baseline, target_names = type_labels))
```

```
Training set
              precision    recall  f1-score   support

          ai       0.88      0.95      0.91       186
   classical       0.99      0.97      0.98       712

    accuracy                           0.96       898
   macro avg       0.94      0.96      0.95       898
weighted avg       0.96      0.96      0.96       898

Test set
              precision    recall  f1-score   support

          ai       0.72      0.77      0.74        47
   classical       0.94      0.92      0.93       178

    accuracy                           0.89       225
   macro avg       0.83      0.84      0.84       225
weighted avg       0.89      0.89      0.89       225
```

After applying the upsampling, the model's performance improved, the test accuracy increased 87% to 89%, while the AI class F1-score improved from 0.55 to 0.74. Most notably, the recall for AI abstracts increased from 0.38 to 0.77, indicating that the model is now able to identify the majority of relevant AI papers. ALthough AI precision decreased to 0.72, this trade off is acceptable for the Researchly application

where retrieving relevant research articles is more important than missing them and overall upsampling effectively relieved the class imbalance and produduced a more balanced classifier that performs well on both AI and Classical research abstracts.

2.3

In [18]:
```
import joblib
from sklearn.ensemble import RandomForestClassifier

#Train best model
RF_Model = RandomForestClassifier(n_estimators = 300, random_state = 42)
RF_Model.fit(X_train, y_train)

#Save model
joblib.dump(RF_Model, "RF_Model.pkl")
joblib.dump(tfidf, "vectorizer.pkl")
```

Out[18]:
```
['vectorizer.pkl']
```

In [19]:
```
%%writefile app.py
import streamlit as st
import joblib

#Load model and vectorizer
model = joblib.load("RF_Model.pkl")
vectorizer = joblib.load("vectorizer.pkl")

st.title("Researchly Research Abstract Classifier")
st.write(""" This app classifiees research abstracts as either AI-related or classical research.""")

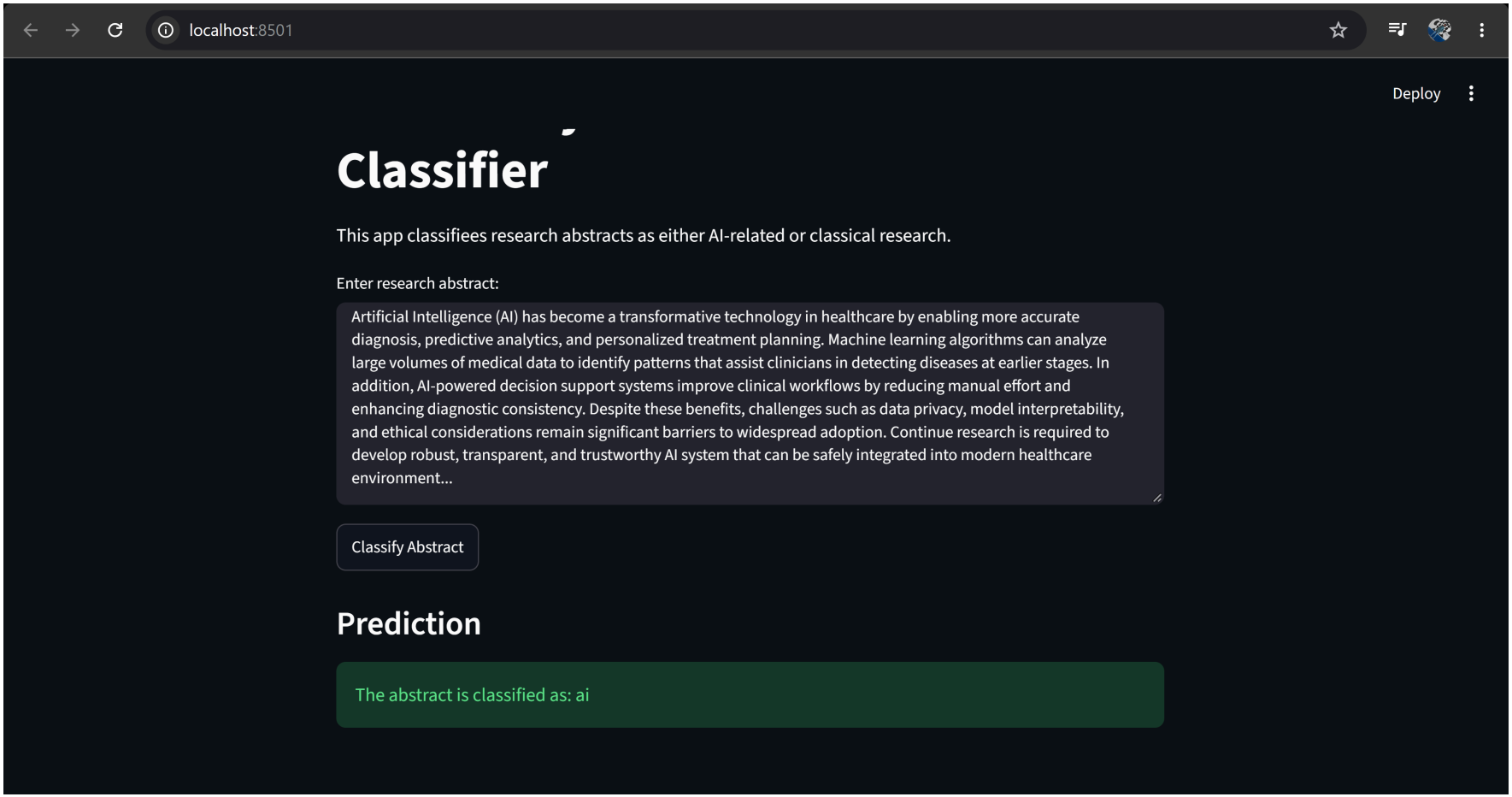
abstract = st.text_area("Enter research abstract:",
                        height = 200)

if st.button("Classify Abstract"):
    if abstract.strip() == "":
        st.warning("Please enter an abstract.")
    else:
        #Convert text into numerical features
        abstract_vector = vectorizer.transform([abstract])
```

```
        #predict class
        prediction = model.predict(abstract_vector)[0]

        st.subheader("Prediction")
        st.success(f"The abstract is classified as: {prediction}")
```

```
Overwriting app.py
```



In [20]:
```
import ipywidgets as widgets
from IPython.display import display, HTML
```

```
Label_selector = {"ai": "artificial intelligence", "classical": "classical research"}

def classify_abstract(raw_text):
    clean = denoise_text(raw_text)
    vec = tfidf.transform([clean])
    pred = RF_Model.predict(vec)[0]
    proba = RF_Model.predict_proba(vec)[0]
    class_idx = list(RF_Model.classes_).index(pred)
    confidence = proba[class_idx]
    return pred, Label_selector[pred], confidence

#Interface
title = widgets.HTML(
    "<h1 style = 'font-weight:800'>Researchly Research Abstract Classifier</h1>"
    "<p>This app classifies research abstracts as either AI-related or classical research.</p>")

label = widgets.HTML("<b>Enter research abstract</b>")
text_area = widgets.Textarea(
    placeholder = "Write or paster a research abstract here...",
    layout = widgets.Layout(width = "100%", height = "160px"))

classify_button = widgets.Button(description = "Classify Abstract")
output = widgets.Output()

def on_classify_click(b):
    with output:
        output.clear_output()
        text = text_area.value.strip()
        if not text:
            display(HTML("<p>Please enter an abstract to classifiy.</p>"))
            return
        pred, disp_label, confidence = classify_abstract(text)
        colour = "#15803d" if pred == "ai" else "#92400e"
        bg = "#dcfce7" if pred == "ai" else "#fef3c7"
        display(HTML(
            f"<h2 style = 'font-weight:800'>Prediction</h2>"
            f"<div style = 'background:{bg}; color:{colour};"
            f"border-radius: 10px; font-weight:500'> The abstract is classified as: {disp_label}</div>"))
```